

Validación de la plataforma ROOT_LABS

Campaña completa de pruebas unitarias, de humo, de estrés y de carga sobre la plataforma CTF de contenedores dinámicos. Cada resultado de este informe procede de una ejecución real registrada en pruebas/resultados/, contra el Docker Engine del host y contenedores verdaderos.

Fecha de ejecución	Revisión probada	Host
20 · 09 · 2026	33cf672	WSL2 · kernel 6.6.87.2
Recursos	Docker Engine	Servidor
12 CPU · 7.785 MB RAM	28.5.1	Daphne 4.2.3 · Django 5.2.17

01 Resumen de resultados

UNITARIAS 84 / 84 8,35 s · 5 módulos	HUMO 10 / 10 Docker real	ESTRÉS 8 / 8 con comparativo
CARGA 20 / 20 usuarios concurrentes		

Las 122 verificaciones ejecutadas pasaron. Además, la campaña detectó **tres defectos reales** que no estaban documentados en el proyecto y que se detallan en la sección 06: el límite de memoria efectivo es el doble del declarado, el techo de estudiantes concurrentes lo impone la red y no la RAM, y la destrucción de instancias desperdicia cinco segundos por contenedor.

Encontrar defectos no contradice los resultados: las pruebas se diseñaron para atacar el sistema, y un sistema que se deja medir es el que puede corregirse.

02 Metodología

La validación se organizó en cuatro niveles, de menor a mayor contacto con el entorno real. Los dos primeros responden «¿está bien construido?»; los dos últimos, «¿aguanta?».

NIVELES DE PRUEBA

NIVEL	QUÉ COMPRUEBA	ENTORNO	CASOS
Unitaria	Lógica del orquestador aislada: catálogo, banderas, modelos, API y watchdog	Docker sustituido por dobles de prueba	84
Humo	Ciclo de vida completo de una instancia contra el motor real	Docker Engine real	10
Estrés	Resistencia del aislamiento ante ataques desde dentro del contenedor	Docker real · dos configuraciones	8
Carga	Comportamiento con 20 estudiantes simultáneos de extremo a extremo	Daphne + 20 contenedores reales	7 fases

En las pruebas de estrés cada ataque se ejecuta **dos veces**: primero sobre un contenedor sin los límites del sistema y después sobre uno con la configuración real de la plataforma. Así la mejora queda medida, no afirmada — es el origen de los cuadros comparativos de este informe.

03 Pruebas unitarias

Suite nueva de 84 casos repartidos en cinco módulos dentro de ctf/tests/. No requiere Docker: las llamadas al motor de contenedores se sustituyen por dobles de prueba, de modo que la suite corre en cualquier equipo y puede integrarse en un flujo de integración continua.

ejecución completa

pruebas/resultados/01-unitarias.txt

```
$ python manage.py test ctf.tests -v 2

Creating test database for alias 'default'...
Found 84 test(s).

[... ejecución de los cinco módulos ...]

-----
Ran 84 tests in 8.351s

OK
Destroying test database for alias 'default'...
System check identified no issues (0 silenced).
```

LOS CINCO MÓDULOS

MÓDULO	RIESGO QUE CUBRE	CASOS
test_challenges	Que un estudiante apruebe sin resolver el reto	20
test_models	Que un estudiante acapare recursos o falsee su progreso	13
test_views	Que la API deje recursos colgados o acepte peticiones indebidas	23
test_seguridad	Que una regresión elimine un límite sin que nadie lo note	16
test_watchdog	Que los contenedores abandonados no se liberen nunca	12

test_challenges

Catálogo de retos y banderas dinámicas

20 CASOS

Qué resuelve. Cada estudiante recibe una bandera propia derivada de la clave secreta del servidor, su identificador y el reto. Estas pruebas garantizan que compartir la bandera con un compañero no le sirva de nada, que una bandera filtrada del código no valide, y que el catálogo que viaja al navegador nunca contenga la respuesta.

 test_challenges · 20 casos

```
$ python manage.py test ctf.tests.test_challenges -v 2
```

```
CatalogoRetosTest - integridad del catálogo
```

```
test_el_catalogo_tiene_diez_retos ... ok
test_cada_reto_cubre_una_categoria_owasp_distinta ... ok
test_todos_los_retos_declaran_los_campos_obligatorios ... ok
test_todas_las_dificultades_son_validas ... ok
test_el_reto_por_defecto_existe_en_el_catalogo ... ok
test_get_challenge_devuelve_none_si_no_existe ... ok
```

```
BanderaDinamicaTest - generación de la bandera
```

```
test_la_bandera_es_determinista_para_el_mismo_usuario ... ok
test_la_bandera_cambia_entre_usuarios ... ok
test_la_bandera_cambia_entre_retos_del_mismo_usuario ... ok
test_la_bandera_respeto_el_formato_publicado ... ok
test_todos_los_retos_generan_banderas_distintas_entre_si ... ok
```

```
ValidacionBanderaTest - quién puede canjearla
```

```
test_acepta_la_bandera_propia ... ok
test_rechaza_la_bandera_de_otro_estudiante ... ok
test_rechaza_la_bandera_correcta_de_otro_reto ... ok
test_rechaza_si_no_se_identifica_al_usuario ... ok
test_rechaza_bandera_vacia ... ok
test_rechaza_reto_inexistente ... ok
test_ignora_espacios_alrededor_de_la_bandera ... ok
```

```
ExposicionDeDatosTest - qué se publica al frontend
```

```
test_list_challenges_no_expone_ninguna_bandera ... ok
test_list_challenges_devuelve_todos_los_retos_con_xp ... ok
```

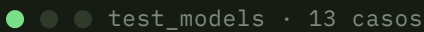
```
Ran 20 tests - OK
```

test_models

Invariantes de la base de datos

13 CASOS

Qué resuelve. Las reglas que no pueden depender de que el código «se acuerde» de comprobarlas: un estudiante, una instancia; una sola fila de configuración; un reto resuelto una sola vez por persona. Aquí se verifica que la propia base de datos las rechace, no la aplicación.

test_models · 13 casos

```
$ python manage.py test ctf.tests.test_models -v 2
```

```
InstanceTest – una instancia activa por estudiante
```

```
test_un_usuario_no_puede_tener_dos_instancias ... ok
test_dos_usuarios_distintos_pueden_tener_instancia_a_la_vez ... ok
test_el_contador_de_intentos_arranca_en_cero ... ok
test_borrar_el_usuario_borra_su_instancia ... ok
```

```
PlatformSettingsTest – configuración en caliente
```

```
test_actual_crea_la_fila_si_no_existe ... ok
test_actual_devuelve_siempre_la_misma_fila ... ok
test_limite_de_tiempo_por_dificultad ... ok
test_dificultad_desconocida_usa_el_limite_basico ... ok
test_el_cambio_se_persiste_sin_reiniciar ... ok
```

```
SolvedChallengeTest – progreso y XP
```

```
test_no_se_puede_registrar_dos_veces_el_mismo_reto ... ok
test_dos_usuarios_pueden_resolver_el_mismo_reto ... ok
```

```
InstanceDestructionNoticeTest – aviso del motivo
```

```
test_los_tres_motivos_estan_disponibles ... ok
test_se_puede_registrar_un_aviso ... ok
```

```
Ran 13 tests – OK
```

Qué resuelve. Los dos fallos más caros de esta plataforma: que un anónimo alcance la API, y que un error a mitad del despliegue deje un contenedor vivo sin registro que lo reclame. Incluye el límite de cinco intentos de bandera, que corta la fuerza bruta.

test_views · 23 casos

```
$ python manage.py test ctf.tests.test_views -v 2
```

```
AutenticacionRequeridaTest - nada es público
```

```
test_get_sin_sesion_no_devuelve_datos ... ok
```

```
test_post_sin_sesion_no_ejecuta_nada ... ok
```

```
test_la_terminal_no_es_publica ... ok
```

```
CatalogoAPITest - qué devuelve /api/challenges/
```

```
test_devuelve_los_diez_retos ... ok
```

```
test_no_filtra_ninguna_bandera_al_frontend ... ok
```

```
DespliegueInstanciaTest - POST /api/instance/start/
```

```
test_despliega_y_registra_la_instancia ... ok
```

```
test_inyecta_la_bandera_del_estudiante_en_el_contenedor ... ok
```

```
test_la_red_es_exclusiva_del_usuario ... ok
```

```
test_rechaza_una_segunda_instancia_del_mismo_usuario ... ok
```

```
test_rechaza_un_reto_inexistente ... ok
```

```
test_si_docker_falla_no_deja_recursos_colgados ... ok
```

```
DetenerInstanciaTest - POST /api/instance/stop/
```

```
test_destruye_contenedor_red_y_fila ... ok
```

```
test_sin_instancia_activa_responde_404 ... ok
```

```
test_si_docker_falla_conserva_la_fila ... ok
```

```
EnvioBanderaTest - POST /api/challenges/submit/
```

```
test_bandera_correcta_registra_el_reto_resuelto ... ok
```

```
test_bandera_de_otro_estudiante_no_sirve ... ok
```

```
test_bandera_vacia_se_rechaza_sin_gastar_intento ... ok
```

```
test_resolver_dos_veces_no_duplica_el_progreso ... ok
```

```
test_cada_fallo_descuenta_un_intento ... ok
```

```
test_al_quinto_fallo_destruye_la_instancia ... ok
```

```
test_redespargar_reinicia_el_contador_de_intentos ... ok
```

```
test_reto_inexistente_responde_404 ... ok
```

```
test_json_invalidado_se_rechaza ... ok
```

```
Ran 23 tests - OK
```

test_seguridad

Límites enviados al kernel y panel de administración

16 CASOS

Qué resuelve. Es la red de seguridad contra regresiones. Intercepta el JSON que el orquestador envía al socket de Docker y comprueba que lleva memoria, CPU, procesos, descriptores, capacidades y montajes. Si alguien elimina PidsLimit o la red interna, la suite falla antes de que el cambio llegue al aula.

```
● ● ● test_seguridad · 16 casos
```

```
$ python manage.py test ctf.tests.test_seguridad -v 2
```

```
LimitesDeRecursosTest – lo que el contenedor podrá consumir
```

```
test_declara_limite_de_memoria ... ok
test_declara_limite_de_cpu ... ok
test_declara_limite_de_procesos ... ok
test_declara_limite_de_descriptores_de_archivo ... ok
test_el_contenedor_usa_la_red_del_usuario ... ok
```

```
EndurecimientoContenedorTest – lo que NO podrá hacer
```

```
test_descarta_todas_las_capacidades_del_kernel ... ok
test_solo_devuelve_las_capacidades_minimas_para_sudo ... ok
test_tmp_esta_en_memoria_sin_ejecucion_ni_setuid ... ok
test_usa_un_init_que_cosecha_procesos_zombis ... ok
test_acota_el_tamano_de_los_logs_en_disco ... ok
```

```
AislamientoDeRedTest – la red de cada estudiante
```

```
test_la_red_es_interna_sin_salida_a_internet ... ok
test_la_red_es_de_tipo_bridge ... ok
```

```
PanelAdministracionTest – separación de roles
```

```
test_un_estudiante_no_accede_a_la_configuracion ... ok
test_un_estudiante_no_puede_destruir_instancias_ajenas ... ok
test_un_anonimo_tampoco_accede ... ok
test_el_profesor_si_accede_a_la_configuracion ... ok
```

```
Ran 16 tests – OK
```

test_watchdog

Destrucción automática de instancias

12 CASOS

Qué resuelve. El watchdog es lo único que impide que los contenedores se acumulen hasta agotar el servidor. Se verifican las dos condiciones de expiración por separado —abandono y vida máxima según dificultad— y, sobre todo, la regla crítica: si Docker falla al destruir, la fila *no* se borra, porque borrarla dejaría un contenedor huérfano que nadie volvería a reclamar.

 test_watchdog · 12 casos

```
$ python manage.py test ctf.tests.test_watchdog -v 2
```

```
Instancias que deben sobrevivir
```

```
test_no_destruye_una_instancia_activa_y_reciente ... ok
```

```
test_no_destruye_justo_antes_del_umbral ... ok
```

```
Expiración por abandono
```

```
test_destruye_por_inactividad ... ok
```

```
test_el_aviso_por_inactividad_lleva_ese_motivo ... ok
```

```
Expiración por vida máxima
```

```
test_destruye_por_vida_maxima_aunque_este_activa ... ok
```

```
test_la_vida_maxima_depends_de_la_dificultad ... ok
```

```
test_reto_fuera_del_catalogo_usa_el_limite_global ... ok
```

```
test_deja_aviso_con_el_motivo_correcto ... ok
```

```
Resistencia a fallos de Docker
```

```
test_si_docker_falla_no BORRA la fila ... ok
```

```
test_un_fallo_no impide limpiar las demas ... ok
```

```
Barrido masivo
```

```
test_barre_varias_instancias_en_una_pasada ... ok
```

```
test_barrido_sin_instancias_no falla ... ok
```

```
Ran 12 tests - OK
```

04 Pruebas de humo

Recorren el ciclo de vida completo de una instancia ejercitando el cliente Docker propio del proyecto (ctf/docker_client.py) contra el motor real. Sin dobles de prueba.

LAS DIEZ PRUEBAS, POR CATEGORÍA

CATEGORÍA	PREGUNTA QUE RESPONDE	PRUEBAS
Orquestación de recursos	¿Puede el sistema crear el entorno de un estudiante desde cero?	H-01 · H-02 · H-04
Aplicación de límites	¿Los límites declarados llegaron de verdad al kernel?	H-05
Diseño del reto	¿La bandera es personal, inaccesible sin explotar y obtenible al explotar?	H-03 · H-06 · H-07
Consola interactiva	¿Puede abrirse la terminal dentro del contenedor?	H-08
Supervisión	¿El panel del profesor ve los contenedores reales?	H-09
Liberación	¿El host queda sin residuos al terminar?	H-10

```
pruebas de humo pruebas/resultados/02-humo.txt

$ python pruebas/humo.py

Docker Engine: real | Dobles de prueba: ninguno

[H-01] El cliente Docker propio crea una red aislada
$ docker_client.create_network('ctf-net-humo-1789949475')
Id de red: acb1b1e9250c
Tiempo: 0.036 s
Inspeccion: Internal=true Driver=bridge
>> RESULTADO: OK - red sin salida a internet

[H-04] Arranca el contenedor
$ docker_client.start_container(cont)
Tiempo: 0.282 s
Estado del contenedor: running
>> RESULTADO: OK

[H-05] Los limites llegaron al kernel (vistos desde dentro)
$ docker exec <cont> cat /sys/fs/cgroup/memory.max /sys/fs/cgroup/pids.max
memory.max = 268435456 (esperado 268435456)
pids.max = 64 (esperado 64)
cpu.max = 50000 100000 (esperado = 0.5 CPU)
>> RESULTADO: OK - cgroups aplicados por el kernel

[H-06] La bandera NO es legible por el usuario sin privilegios
$ docker exec -u retador <cont> cat /app/pedidos.json
cat: can't open '/app/pedidos.json': Permission denied
Codigo de salida: 1
>> RESULTADO: OK - hay que explotar la falla para obtenerla

[H-07] La bandera SI es accesible explotando el reto (via sudo)
$ docker exec -u retador <cont> sudo /usr/bin/python3 /app/pedidos.py 1337
{
  "cliente": "admin",
  "producto": "Acceso VIP",
  "flag": "FLAG{idor_12f669e21f3b8de1}"
}
>> RESULTADO: OK - el reto es resoluble

[H-10] Destruccion completa: contenedor y red
$ docker_client.destroy_instance(cont, red)
Tiempo: 5.534 s
Contenedor restante: ninguno
Red restante: ninguna
>> RESULTADO: OK - sin residuos en el host

=====
Total: 10 | Exitosas: 10 | Fallidas: 0
```

Las pruebas H-06 y H-07 son el par que valida el diseño pedagógico: el mismo archivo es ilegible con cat y accesible a través del programa vulnerable. Confirma que la única vía a la bandera es explotar la falla, no leer un archivo.

El tiempo de H-10 (5,53 s) quedó marcado como anomalía y derivó en el hallazgo 3.

05 Pruebas de estrés

Ocho ataques ejecutados desde dentro de un contenedor de estudiante, cada uno en dos configuraciones. Las capturas siguientes muestran los tres casos más representativos; el transcript completo está en pruebas/resultados/03-estres.txt.

LOS OCHO ATAQUES, POR CATEGORÍA

CATEGORÍA	AMENAZA QUE REPRESENTA	PRUEBAS
Contención de recursos	Un estudiante deja al resto sin servicio consumiendo procesos, memoria, CPU o descriptores del host	E-01 · E-02 E-03 · E-07
Aislamiento de red	El reto se usa como trampolín para atacar internet o el entorno de otro estudiante	E-04 · E-05
Privilegios	Root dentro del contenedor conserva capacidades del kernel que el reto no necesita	E-06
Verificación integral	Los límites funcionan sueltos pero alguno se pierde al aplicarlos todos juntos	E-08

E-01 Bomba de procesos**CONTENIDO**

Un proceso intenta crear 200 hijos. Sin límite lo consigue; con PidsLimit el kernel deniega la llamada fork() al llegar al cupo.

```
E-01 · bomba de procesos

--- ANTES · contenedor sin PidsLimit ---
$ docker run --rm ctf-base:latest python3 -c '<fork x200>'
procesos creados: 200

--- DESPUES · contenedor con --pids-limit=64 ---
$ docker run --rm --pids-limit=64 ctf-base:latest python3 -c '<fork x200>'
bloqueado por el kernel: [Errno 11] Resource temporarily unavailable
procesos creados: 62

COMPARATIVO | sin limite: 200 procesos -> con limite: 62 procesos
```

E-05 Un estudiante intenta alcanzar al otro**AISLADO**

El alumno B expone un servicio. El alumno A intenta conectarse, primero compartiendo red y después con el modelo real de la plataforma: una red interna por usuario.

```
E-05 · aislamiento entre estudiantes

Alumno B expone un servicio en 172.22.0.2:8000

--- ANTES · ambos alumnos en la MISMA red ---
$ docker run --rm --network ctf-estres-abierta ... <conectar a IP del alumno B>
ALCANZADO: el alumno B es visible desde aqui

--- DESPUES · cada alumno en su PROPIA red interna ---
$ docker run --rm --network ctf-estres-red-b ... <conectar a IP del alumno B>
INALCANZABLE: OSError [Errno 101] Network unreachable
```

E-08 Perfil completo de la plataforma**VERIFICADO**

Inventario de todos los límites aplicados simultáneamente, leídos desde dentro del contenedor a través de la interfaz de cgroups del kernel.

E-08 · perfil aplicado

```
$ docker run --rm --memory=256m --cpus=0.5 --pids-limit=64 \
  --cap-drop=ALL --cap-add=SETUID --cap-add=SETGID \
  --tmpfs /tmp:rw,noexec,nosuid,size=64m \
  --ulimit nofile=1024:2048 --init ctf-base:latest sh -c '...'

memoria max : 268435456
cpu max      : 50000 100000
pids max     : 64
descriptores: 1024
capacidades : 00000000000000c0
montaje tmp  : tmpfs /tmp tmpfs rw,nosuid,nodev,noexec,relatime,size=65536k
```

CUADRO COMPARATIVO · LAS OCHO PRUEBAS DE ESTRÉS

ATAQUE	SIN LOS LÍMITES	CON LA PLATAFORMA	QUÉ LO IMPIDE
E-01 Bomba de procesos	200 procesos	62 procesos	cgroup pids
E-02 Agotar memoria	400 MB asignados	OOM, código 137	cgroup memory
E-03 Saturar CPU	100,25 %	50,02 %	planificador CFS
E-04 Salir a internet	conectado a 1.1.1.1	Network unreachable	red Internal
E-05 Alcanzar a otro alumno	servicio visible	Network unreachable	una red por usuario
E-06 Privilegios de root	a80425fb · 14 caps	000000c0 · 2 caps	CapDrop: ALL
E-07 Descriptores de archivo	1 048 576	1 024	Ulimits nofile
E-08 Perfil completo	—	6 / 6 aplicados	cgroups + namespaces

En los ocho casos quien detiene el ataque es el kernel, no código de la aplicación. Es la diferencia entre un límite que se comprueba y uno que se concede: el planificador y los cgroups deciden durante la llamada al sistema, antes de entregar el recurso.

06 Prueba de carga · 20 estudiantes

Veinte estudiantes simulados se autentican, despliegan su instancia, abren la consola, ejecutan un comando y liberan los recursos. Todo contra el servidor Daphne real y veinte contenedores verdaderos.

LAS SIETE FASES DE LA PRUEBA

FASE	QUÉ MIDE	RESULTADO
C-00	Preparación de cuentas y estado inicial del servidor	20 cuentas · 1 hilo en reposo
C-01	Autenticación concurrente	20 / 20 en 1,21 s
C-02	Oleada de despliegue de instancias	20 / 20 en 7,79 s
C-03	Consolas WebSocket simultáneas con comando real	20 / 20 · 1,63 s de media
C-04	Hilos del servidor consumidos por las consolas	0 bloqueados en E/S
C-05	Consumo real de recursos del host bajo carga	~665 KiB por contenedor
C-06	Liberación y comprobación de residuos	20 / 20 · 0 residuos

```
prueba de carga                                     pruebas/resultados/04-carga.txt

$ python pruebas/carga.py 20

[C-02] DESPLIEGUE CONCURRENTES DE 20 INSTANCIAS
  Instancias creadas: 20 / 20
  Tiempo total de la oleada: 7.79 s
  Latencia por instancia -> min 1.71 s | media 5.71 s | max 7.89 s
  Filas en base de datos: 20   Contenedores reales en Docker: 20

[C-03] CONSOLAS WEBSOCKET SIMULTANEAS
  Consolas que ejecutaron el comando: 20 / 20
  Respuesta por consola -> min 1.31 s | media 1.63 s | max 1.88 s

[C-04] CONSUMO DE HILOS DEL SERVIDOR (metrica critica)
$ for t in /proc/7936/task/*; do cat $t/wchan; done | sort | uniq -c

T1 - durante el arranque de las consolas:
    17  futex_wait_queue      (ociosos)
     1  do_epoll_wait
    bloqueados en E/S de socket: 0

T2 - con las 20 consolas ya abiertas y en reposo:
    17  futex_wait_queue      (ociosos)
     1  do_epoll_wait
    bloqueados en E/S de socket: 0

Hilos totales: reposo 1 -> arranque 18 -> consolas abiertas 18

[C-06] LIBERACION DE RECURSOS
  Instancias detenidas: 20 / 20
  Tiempo total: 12.40 s
  Filas en base de datos tras la limpieza: 0
  Contenedores restantes: 0
```

El dato que cierra el caso del techo de consolas

El host tiene 12 CPU, de modo que el pool de hilos de Python admite como máximo **16** — exactamente el techo histórico documentado en el proyecto. Si cada consola consumiera un hilo propio, 20 consolas serían imposibles: el pool se agotaría en la decimosexta.

Se midieron 20 consolas simultáneas funcionando, con **cero** hilos bloqueados en entrada/salida de socket, tanto durante el arranque como con todas las terminales abiertas en reposo. Los 17 hilos presentes están en `futex_wait_queue`: ociosos, esperando trabajo, no bloqueados.

CUADRO COMPARATIVO · CONSOLAS SIMULTÁNEAS

MEDICIÓN	DISEÑO ANTERIOR (UN HILO POR CONSOLA)	DISEÑO ACTUAL (MEDIDO HOY)
Consolas que ejecutan el comando	0 / 20	20 / 20
Estado de los hilos del servidor	bloqueados en E/S	ociosos
Hilos bloqueados en socket	hasta 16 (pool lleno)	0
Techo por diseño	~16 consolas	sin techo de hilos
Despliegue de instancia bajo carga	timeout a los 30 s	5,71 s de media

07 Defectos encontrados

Tres defectos que las pruebas revelaron y que no estaban documentados en el proyecto. Ninguno compromete el aislamiento; los tres afectan la capacidad o el aprovechamiento de recursos.

HALLAZGO 1 · SEVERIDAD ALTA · CAPACIDAD

El techo de estudiantes lo impone la red, no la memoria

Al elevar la prueba de carga a 30 estudiantes, seis despliegues fallaron con error 500. La causa no es memoria ni CPU: Docker agota sus rangos de direcciones predefinidos alrededor de la red número 24.

```
$ docker network create --internal ctf-pool-test-24
Error response from daemon: all predefined address pools have been fully subnetted
```

La documentación del proyecto estima «50-60 contenedores simultáneos» a partir de la memoria. El límite real medido es de **24 a 31 estudiantes**, porque cada uno consume una red completa. Corrección en `/etc/docker/daemon.json`:

```
{ "default-address-pools": [ { "base": "10.200.0.0/16", "size": 24 } ] }
```

Con máscara /24 sobre esa base se obtienen 256 redes, y el techo vuelve a depender de la memoria del host.

HALLAZGO 2 · SEVERIDAD MEDIA · RECURSOS

El límite de memoria efectivo es el doble del declarado

La plataforma declara Memory pero no MemorySwap. Docker concede entonces una cantidad de swap igual al límite de RAM, así que cada contenedor puede llegar a 512 MB en lugar de los 256 MB documentados. Un contenedor con límite de 256 MB asignó 400 MB sin problema; declarando el tope de swap, el kernel lo detuvo con código 137.

```
# ctf/docker_client.py · HostConfig
    "Memory": mem_limit_bytes,
+   "MemorySwap": mem_limit_bytes,    # sin esto, Docker concede otro tanto de swap
```

HALLAZGO 3 · SEVERIDAD MEDIA · RENDIMIENTO

Cada destrucción desperdicia cinco segundos

El proceso principal del contenedor es un shell que no termina al recibir SIGTERM, así que Docker espera el tiempo de gracia completo antes de forzar el cierre. Medido de forma aislada: **5,30 s** con el valor actual frente a **0,37 s** sin espera — catorce veces más rápido. Explica los 12,4 s que tardó liberar las veinte instancias de la prueba de carga.

```
# ctf/docker_client.py
- def stop_container(container_id: str, timeout: int = 5) -> None:
+ def stop_container(container_id: str, timeout: int = 0) -> None:
```

08 Reproducibilidad

Toda la campaña se vuelve a ejecutar con cuatro órdenes. Los transcripts se guardan en pruebas/resultados/.

● ● ● reproducir la campaña

```
# 1 · unitarias — no necesitan Docker
python manage.py test ctf.tests -v 2

# 2 · humo — requiere Docker y las imágenes de los retos
python pruebas/humo.py

# 3 · estrés — comparativo antes/después
bash pruebas/estres.sh

# 4 · carga — requiere Daphne corriendo
daphne -b 127.0.0.1 -p 8000 ctf_platform.asgi:application &
python pruebas/carga.py 20
```

-
- 84** casos unitarios en ctf/tests/ — 5 módulos

 - 10** pruebas de humo en pruebas/humo.py — 6 categorías

 - 8** pruebas de estrés en pruebas/estres.sh — 4 categorías

 - 7** fases de carga en pruebas/carga.py

Plataforma ROOT_LABS · Informe de pruebas y validación · Revisión 33cf672 · Ejecutado el 20 de septiembre de 2026 sobre WSL2 con Docker Engine 28.5.1. Todas las cifras proceden de ejecuciones registradas; ninguna es estimada.